

MilkWise — Code Verification Audit

Document purpose: Verify that the equations described in `milkwise-calculation-audit.pdf` are correctly and consistently implemented in all three codebases.

Date: 2026-09-08

Auditor: Kit (agent-app-dev subagent)

§1 Methodology

The source code is the **ground truth**. The audit document (`milkwise-calculation-audit.pdf`) is a description of what the code should do. When the code and the document diverge, the document is wrong — the document must be updated to match the code, not the other way around.

Files examined:

File	Role
<code>varia/baby-milk-tracker/src/lib/calculations.ts</code>	Web app — core math
<code>varia/baby-milk-tracker/src/components/cards/CanTakeCard.tsx</code>	Web app — timeline UI, ghost logic
<code>varia/milkwise/src/lib/calculations.ts</code>	React Native app — core math
<code>varia/milkwise/src/screens/DashboardScreen.tsx</code>	RN app — orchestration, WHO integration
<code>varia/baby-milk-tracker/src/lib/whoGrowth.ts</code>	Web app — WHO model
<code>varia/milkwise/src/lib/whoGrowth.ts</code>	RN app — WHO model

Scope note: EQ-1 through EQ-12 live in `calculations.ts`. EQ-13 through EQ-15 live in `whoGrowth.ts`. EQ-11 (`ghostReadyAt`) is implemented in `CanTakeCard.tsx` (web) and in `calculations.ts` (RN — as the exported `ghostIntakeReadyAtMs` function).

§2 Equation-by-Equation Verification

EQ-1 — Water-to-Formula Conversion

Doc says: Piecewise linear interpolation using a 7-row lookup table; if **waterMl** is below 30 or above 210, the nearest table endpoint is used (no extrapolation).

Code (both implementations):

```
// Below lowest entry – extrapolate from first segment
if (waterMl <= t[0].water) {
  const slope = (t[1].formula - t[0].formula) / (t[1].water -
t[0].water);
  return t[0].formula + slope * (waterMl - t[0].water);
}
// Above highest entry – extrapolate from last segment
if (waterMl >= t[last].water) {
  const slope = (t[last].formula - t[last - 1].formula) /
(t[last].water - t[last - 1].water);
  return t[last].formula + slope * (waterMl - t[last].water);
}
```

Status: ⚠ PARTIAL MATCH

The lookup table values and interpolation logic are correct. However, the boundary behaviour diverges: the doc says "no extrapolation — nearest endpoint is used", but the code **extrapolates** using the slope of the nearest segment. For out-of-range inputs (e.g., **waterMl** = 15 or **waterMl** = 240), the code returns an extrapolated value, not the endpoint value.

In practice, the UI only allows **FORMULA_TABLE** entries as inputs, so this discrepancy has no runtime impact. The doc comment at the top of **calculations.ts** is actually correct ("nearest segment slope is extrapolated") — only the EQ-1 entry in the PDF is wrong.

EQ-2 — Daily Target

Code (both implementations):

```
export function deriveSettings(settings: Settings):
DerivedSettings {
  const dailyTargetMl = settings.weightKg *
settings.mlPerKgPerDay;
```

```
    ...  
  }
```

Status: MATCH

Formula is `weightKg × mlPerKgPerDay` exactly as documented. The doc's note that `weightKg` comes from measured weight or the WHO prediction (EQ-15) is also correctly implemented — both apps pass `effectiveWeightKg` into `deriveSettings`.

EQ-3 — Hourly Rate

Code (both implementations):

```
const hourlyRate = dailyTargetMl / 24;
```

Status: MATCH

EQ-4 — Standard Interval

Code (both implementations):

```
const milkPerBottle =  
  waterToMilk(settings.preferredBottleWaterMl);  
const idealIntervalHours = milkPerBottle / hourlyRate;
```

Status: MATCH

`formulaPerBottle / hourlyRate` exactly as documented.

EQ-5 — Bottle Credit

Code (both implementations):

```
export function bottleCredit(ageHours: number, milkMl: number,  
  hourlyRate: number): number {  
  if (ageHours <= 24) {  
    return milkMl;  
  } else {  
    const decay = hourlyRate * (ageHours - 24);
```

```
    return Math.max(0, milkMl - decay);
  }
}
```

Status: MATCH

Two-phase model exactly as documented. Phase 1: full credit for age $\leq$ 24h. Phase 2: linear decay at **hourlyRate** ml/h until zero.

EQ-6 — Smoothed Intake

Code (both implementations):

```
export function smoothedAtTime(feeds: Feed[], hourlyRate: number,
atMs: number): number {
  return feeds.reduce((sum, f) => {
    const ageHours = (atMs - f.timestamp) / 3_600_000;
    return sum + bottleCredit(ageHours, waterToMilk(f.volume),
hourlyRate);
  }, 0);
}
```

Status: MATCH

Sums **bottleCredit** over all feeds at the given evaluation time. The Frozen Status Principle (§6 of the audit doc) is also implemented: status is evaluated at **lastFeed.timestamp**, not at **now**.

Web app:

```
// page.tsx
const smoothedAt = lastFeed ? lastFeed.timestamp : now;
const { totalMl: smoothedMl } = smoothedEffective(feeds,
effectiveDerived.hourlyRate, ..., smoothedAt);
```

RN app:

```
// DashboardScreen.tsx
const smoothedAt = lastFeed ? lastFeed.timestamp : now;
```

```
const { totalMl: smoothedMl } = smoothedEffective(feeds,
derived.hourlyRate, ..., smoothedAt);
```

EQ-7 — Stomach Load

Doc says: $\text{stomachLoad}(\text{atTime}) = \sum \text{formulaMl}_i \times \exp(-k \times (\text{atTime} - \text{feedTime}_i))$ summing over all recent feeds, with no explicit age cutoff.

Code (both implementations):

```
export function stomachLoad(feeds: Feed[], atMs: number): number {
  const atHours = atMs / 3_600_000;
  return feeds.reduce((sum, f) => {
    const ageHours = atHours - f.timestamp / 3_600_000;
    if (ageHours < 0 || ageHours > 7) return sum; // ←
    UNDOCUMENTED CUTOFF
    return sum + waterToMilk(f.volume) * Math.exp(-STOMACH_K *
ageHours);
  }, 0);
}
```

Status: ⚠ PARTIAL MATCH

The exponential formula is correctly implemented. However, the code includes a **hard 7-hour cutoff** that is not mentioned in the audit doc at all. Feeds older than 7 hours contribute zero to stomach load, regardless of the exponential decay value at that age.

At 7 hours, $\exp(-0.6931 \times 7) \approx 0.8\%$ of the feed remains in the stomach model. This is negligible in practice (a 150 ml feed leaves ≈ 1.2 ml of residual). The cutoff is a reasonable optimisation but the doc should document it. The $\text{ageHours} < 0$ guard (future feeds excluded) is also undocumented but obviously correct.

EQ-8 — Stomach Capacity

Doc says: $\text{stomachCapacity} = \text{formulaPerBottle} / (1 - \exp(-k \times \text{standardInterval}))$; 10% buffer applied when checking readiness.

Code (both implementations):

```
export function stomachCapMilk(preferredBottleWaterMl: number,
hourlyRate: number): number {
```

```

const m0 = waterToMilk(preferredBottleWaterMl);
const SI = m0 / hourlyRate;    // standard interval in hours
const denom = 1 - Math.exp(-STOMACH_K * SI);
if (denom <= 0) return m0 * 4;
return m0 / denom;
}

```

Status: MATCH

Formula exactly matches. The 10% buffer is applied in `stomachReadyAtMs` (EQ-9), as documented.

EQ-8 parameter source — `effectiveWeightKg`:

Both apps correctly pass `effectiveDerived.hourlyRate` (derived from `effectiveWeightKg`, which is the WHO-corrected weight) to all stomach calculations:

- **Web app (`page.tsx`):** `const effectiveDerived = deriveSettings({ ...settings, weightKg: effectiveWeightKg })` → passed as `hourlyRate={effectiveDerived.hourlyRate}` to `CanTakeCard`.
- **RN app (`DashboardScreen.tsx`):** `const derived = deriveSettings({ ...settings, weightKg: effectiveWeightKg })` → `derived.hourlyRate` used everywhere including `stomachCapMilk(settings.preferredBottleWaterMl, derived.hourlyRate)`.

A previous bug passed `settings.weightKg` (unadjusted) instead of `effectiveWeightKg` to stomach calculations. Both implementations are now correct.

EQ-9 — Stomach-Ready Time

Doc says: Closed-form: `stomachReadyAt = now + max(0, -(1/k) × ln((cap×1.1 - bottleFormulaMl) / stomachLoad(now)))`.

Code (both implementations):

```

export function stomachReadyAtMs(...): number {
  const cap = stomachCapMilk(preferredBottleWaterMl, hourlyRate) *
  1.1;
  const loadNow = stomachLoad(feeds, atMs);
  if (loadNow + m_new <= cap) return atMs; // fits now
  const remainder = cap - m_new;
  if (remainder <= 0) {
    const dtHours = -Math.log(NEAR_ZERO_ML / loadNow) / STOMACH_K;

```

```

    return atMs + Math.max(0, dtHours) * 3_600_000;
  }
  const dtHours = -Math.log(remainder / loadNow) / STOMACH_K;
  if (dtHours <= 0) return atMs;
  return atMs + dtHours * 3_600_000;
}

```

Status: MATCH

The closed-form derivation is correctly implemented. The $\text{remainder} \leq 0$ branch (bottle at or above cap, need near-zero load) is a reasonable edge case not explicitly documented but correct. The 10% buffer is applied here as documented.

EQ-10 — Intake-Ready Time (Live)

Doc says: "Returns earliest $T \geq \text{now}$ such that $\text{smoothedIntake}(T) + \text{bottleFormulaMl} \geq \text{dailyTarget}$." The "display rounding boundary" note specifies: $\text{smoothedIntake}(T) \leq \text{dailyTarget} \times 1.005 - \text{bottleFormulaMl} - 0.1$.

Code (both implementations):

```

export function intakeReadyAtMs(...): number {
  const milkMl = waterToMilk(bottleWaterMl);
  const targetBefore = dailyTargetMl - milkMl;          // NOT
dailyTarget×1.005 - milk - 0.1
  const currentSmoothed = smoothedAtTime(feeds, hourlyRate, now);
  const TOLERANCE_ML = 1;
  if (currentSmoothed < dailyTargetMl - TOLERANCE_ML) return
now;    // underfed → now

  // Binary search ...
  let lo = now, hi = T_max;
  for (let i = 0; i < 40; i++) {
    const mid = Math.floor((lo + hi) / 2);
    if (smoothedAtTime(feeds, hourlyRate, mid) > targetBefore) lo
= mid;
    else hi = mid;
    if (hi - lo < 60_000) break;
  }
  return Math.floor((lo + hi) / 2);    // returns MIDPOINT, not hi
}

```

Status: DIVERGENCE (three sub-divergences)

Sub-divergence 1 — Threshold:

The doc's "display rounding boundary" formula ($\text{dailyTarget} \times 1.005 - \text{milkMl} - 0.1$) is **not** what `intakeReadyAtMs` uses. The live function uses $\text{targetBefore} = \text{dailyTargetMl} - \text{milkMl}$ (exact target minus bottle). The 1.005 boundary is used exclusively in `ghostIntakeReadyAtMs` (EQ-11). The doc conflates these two distinct thresholds.

Sub-divergence 2 — Underfed condition:

The doc says "if `smoothedIntake(now)` is already below `dailyTarget...` return `now` immediately." The code's condition is `currentSmoothed < dailyTargetMl - TOLERANCE_ML` (1 ml tolerance). Concretely: if `smoothedIntake(now) = dailyTargetMl - 0.5 ml`, the code does NOT return `now` — it binary-searches. This tolerance prevents showing "wait 30 min" immediately after logging a feed where floating-point gives `smoothedIntake = 1080.0001` vs `targetBefore = 1080.0`.

Sub-divergence 3 — Binary search return value:

The doc does not specify the return value of the binary search. The code returns `Math.floor((lo + hi) / 2)` — the **midpoint** of the final interval. The ghost function (`ghostIntakeReadyAtMs`, EQ-11) returns `Math.ceil(hi / 60_000) * 60_000` — ceiling to next minute. These are structurally different: the live function returns a millisecond-precision midpoint; the ghost function returns a clean minute boundary.

EQ-11 — Ghost-Ready Time (Timeline Display)

Doc says: Uses `lastFeed.timestamp` as reference (not `now`); uses the 1.005 rounding boundary; result ceiled to next full minute.

Web app (`CanTakeCard.tsx` — local `ghostIntakeReadyAtMs`):

```
function ghostIntakeReadyAtMs(feeds, bottleWaterMl, hourlyRate,
dailyTargetMl, refMs): number {
  const milkMl = waterToMilk(bottleWaterMl);
  const target = dailyTargetMl * 1.005 - milkMl - 0.1;    // ←
1.005 boundary ✓
  const current = smoothedAtTime(feeds, hourlyRate, refMs);
  if (current <= target) return refMs;
  // binary search...
  return Math.ceil(hi / 60_000) * 60_000;    // ← ceiled to minute
✓
}

// Called with lastFeed.timestamp as refMs ✓
```

```
const intakeMs = ghostIntakeReadyAtMs(feeds, waterMl, hourlyRate,
dailyTargetMl, lastFeed.timestamp);
```

RN app (**calculations.ts** — exported **ghostIntakeReadyAtMs**):

```
export function ghostIntakeReadyAtMs(feeds, bottleWaterMl,
hourlyRate, dailyTargetMl, refMs): number {
  const milkMl = waterToMilk(bottleWaterMl);
  const target = dailyTargetMl * 1.005 - milkMl - 0.1;    // ←
1.005 boundary ✓
  // binary search...
  return Math.ceil(hi / 60_000) * 60_000;    // ← ceiled to minute
✓
}
```

Status: **MATCH**

Both implementations correctly implement EQ-11 as documented. The 1.005 boundary, the -0.1 fudge, and the ceiling-to-minute return value are all present. Reference time is **lastFeed.timestamp** in both cases.

EQ-12 — readyAt — Timeline Progression

Code (both implementations):

```
const sReady = stomachReadyAtMs(feeds, w, preferredBottleWaterMl,
now, hourlyRate);
const iReady = intakeReadyAtMs(feeds, w, hourlyRate,
dailyTargetMl, now);
const readyAtMs = Math.max(sReady, iReady);
const fitsNow = readyAtMs <= now + 30_000;
```

Status: **MATCH**

readyAt(x) = max(stomachReadyAt(x), intakeReadyAt(x)) exactly as documented. The advised bottle (largest with **fitsNow = true**) is computed identically in both implementations.

Notable difference in canTakeProgression return set (not a doc issue — undocumented behaviour): The web app returns **all** sizes up to preferred+1 (commented "no noise-cut rule"). The RN app applies a **noise-cutting rule**: if any size fits now, it returns only sizes $\geq$ the largest currently-fitting size. See §3 for comparison.

EQ-13 — WHO Z-Score

Code (both `whoGrowth.ts` files):

```
export function computeZScore(weightKg: number, ageMonths: number,
sex: 'M' | 'F'): number {
  const { L, M, S } = lmsAt(ageMonths, sex);
  return (Math.pow(weightKg / M, L) - 1) / (L * S);
}
```

Status: MATCH

LMS formula $[(X/M)^L - 1] / (L \times S)$ exactly as documented, with interpolated LMS parameters.

EQ-14 — Mean Z-Score

Code (both `whoGrowth.ts` files):

```
export function estimateZChannel(weights, dateOfBirthMs, sex):
number | null {
  const zScores = weights.map(w => {
    const ageMonths = (w.timestamp - dateOfBirthMs) / (365.25 / 12
* 86_400_000);
    if (ageMonths < 0 || ageMonths > 24) return null;
    return computeZScore(w.weightKg, ageMonths, sex);
  }).filter((z): z is number => z !== null);
  return zScores.reduce((a, b) => a + b, 0) / zScores.length;
}
```

Status: MATCH

Arithmetic mean of all z-scores exactly as documented.

EQ-15 — Predicted Weight

Code (both `whoGrowth.ts` files):

```
export function predictWeightFromZ(z: number, ageMonths: number,
sex: 'M' | 'F'): number {
```

```
const { L, M, S } = lmsAt(ageMonths, sex);
return M * Math.pow(1 + L * S * z, 1 / L);
}
```

Status: MATCH

Inverse LMS formula $M \times (1 + L \times S \times z)^{(1/L)}$ exactly as documented. The short-circuit rule (≤ 7 days since last measurement $\rightarrow$ use raw measurement, skip WHO) is correctly implemented in both `page.tsx` (web) and `DashboardScreen.tsx` (RN).

§3 Cross-Implementation Differences

Equation	Web App	RN App
EQ-1 waterToMilk	Interpolation + extrapolation at boundaries	Identical
EQ-2 dailyTarget	<code>effectiveWeightKg × mlPerKgPerDay</code>	Identical
EQ-3 hourlyRate	<code>dailyTarget / 24</code>	Identical
EQ-4 standardInterval	<code>milkPerBottle / hourlyRate</code>	Identical
EQ-5 bottleCredit	Two-phase linear model	Identical
EQ-6 smoothedAtTime	Evaluated at <code>lastFeed.timestamp</code> (frozen)	Identical
EQ-7 stomachLoad	7h cutoff, exponential decay	Identical
EQ-8 stomachCapMilk	Uses <code>effectiveDerived.hourlyRate</code>	Uses <code>derived.hourlyRate</code> (same, derived from <code>effectiveWeightKg</code>)
EQ-9 stomachReadyAtMs	Closed-form	Identical
EQ-10 intakeReadyAtMs	In <code>calculations.ts</code> , returns midpoint	In <code>calculations.ts</code> , identical
EQ-11 ghostIntakeReadyAtMs	Local function in <code>CanTakeCard.tsx</code>	Exported from <code>calculations.ts</code>
EQ-12 canTakeProgression	Returns all sizes (no noise-cut)	Returns only sizes $\geq$ largest available now

Equation	Web App	RN App
EQ-13 computeZScore	In <code>whoGrowth.ts</code>	Identical file
EQ-14 estimateZChannel	In <code>whoGrowth.ts</code>	Identical file
EQ-15 predictWeightKg	In <code>whoGrowth.ts</code> ; 7-day short-circuit in <code>page.tsx</code>	Identical file; 7-day short-circuit in <code>DashboardScreen.tsx</code>
WHO tables	LMS constants in <code>whoGrowth.ts</code>	Byte-for-byte identical

§4 Notable Findings

F-1 — EQ-7 stomachLoad: Undocumented 7-Hour Age Cutoff [⚠](#)

Location: Both `calculations.ts` files, `stomachLoad` function.

The code contains `if (ageHours < 0 || ageHours > 7) return sum` — a hard cutoff that excludes feeds older than 7 hours from stomach load computation. This is not mentioned anywhere in the audit document.

At 7 hours, $\exp(-0.6931 \times 7) \approx 0.008$ — only 0.8% of the original feed volume remains in the exponential model. For a 200 ml feed, this is ≈ 1.6 ml. The cutoff is practically negligible but semantically meaningful: the exponential function never truly reaches zero, so without this cutoff, a feed given 24 hours ago would still contribute ~ 0.000001 ml to the stomach load.

Recommendation: Add to EQ-7 documentation: "Feeds older than 7 hours are excluded from the sum (cutoff). At $t=7h$, the remaining exponential load is $< 1\%$ for any bottle size."

F-2 — EQ-10 intakeReadyAtMs: Three Documentation Gaps

This is the most significant divergence between doc and code.

2a — Wrong threshold documented:

The doc's "Display rounding boundary" subsection implies that `intakeReadyAtMs` uses `dailyTarget × 1.005 - milkMl - 0.1` as its threshold. The code uses `dailyTargetMl - milkMl` (exact). The 1.005 boundary is only used in `ghostIntakeReadyAtMs`.

The doc's "Notes" section conflates the live function (EQ-10) with the ghost function (EQ-11). These are separate functions with different thresholds serving different purposes.

2b — Underfed condition tolerance:

The doc says "if `smoothedIntake(now)` is already below `dailyTarget...`

return `now` immediately". The code has `currentSmoothed < dailyTargetMl - TOLERANCE_ML` where `TOLERANCE_ML = 1`. A baby with smoothed intake of `dailyTargetMl - 0.3 ml` (just barely below target) will NOT get `now` — the code will binary-search for a small positive delay. The 1 ml tolerance is deliberate (absorbs float drift) but is undocumented.

2c — Binary search return value:

The code returns `Math.floor((lo + hi) / 2)` — the midpoint of the final search interval, with millisecond precision. The doc does not specify the return value. The ghost function returns `Math.ceil(hi / 60_000) × 60_000` (ceiling to next minute). These produce different timestamps for the same input, by design.

F-3 — EQ-12 canTakeProgression: Noise-Cut Rule Differs Between Implementations

Web app (no noise-cut rule): Returns all candidate sizes from 30 ml up to preferred+1. If sizes 30, 60, 90 all fit now, all three are shown.

RN app (noise-cut rule applied): Returns only sizes $\geq$ the largest size currently fitting. If 30, 60, 90 all fit now, only 90 (and any larger future sizes) are returned. Smaller sizes that also fit are suppressed.

This is an intentional UX difference — the RN app targets a simpler list display where showing 3 "fits now" entries would be confusing. The web app uses a timeline visual where showing all available sizes is informative. However, the audit document does not document this divergence and describes only one behaviour without clarifying which implementation it describes.

F-4 — EQ-11 ghostIntakeReadyAtMs: Location Asymmetry

Web app: `ghostIntakeReadyAtMs` is a **private local function** inside `CanTakeCard.tsx` (the UI component). It is not exported from `calculations.ts`.

RN app: `ghostIntakeReadyAtMs` is an **exported function** in `calculations.ts`.

The implementations are functionally identical (same algorithm, same threshold, same ceiling return). But the web app's function is not accessible to other components — if any other web component ever needs ghost timing, it would need to duplicate the function or it would need to be moved to `calculations.ts`. The RN architecture is cleaner in this respect.

Recommendation: Move `ghostIntakeReadyAtMs` from `CanTakeCard.tsx` into `calculations.ts` (web) to match the RN architecture and ensure a single source of truth.

F-5 — EQ-1 Boundary Behaviour Documentation Error ⚠

The audit doc states: "If `waterMl` is below 30 or above 210, the nearest table endpoint is used (no extrapolation)."

The code does the opposite: it **extrapolates** using the slope of the nearest segment. This is correctly documented in the `calculations.ts` file header comment ("For volumes outside the table range, the nearest segment slope is extrapolated") but incorrectly stated in the PDF.

Since the UI constrains inputs to `FORMULA_TABLE` entries, this has no runtime impact. But the PDF is factually wrong about boundary behaviour.

F-6 — WHO Model: Byte-for-Byte Identical

`whoGrowth.ts` is identical between web and RN — same LMS table values, same `lmsAt` interpolation, same `computeZScore`, `predictWeightFromZ`, `estimateZChannel`, and `predictWeightKg` functions. No risk of drift.

F-7 — EQ-8 stomachCapacity: effectiveWeightKg Correctly Applied

A previous bug passed `settings.weightKg` (unadjusted) instead of the WHO-corrected `effectiveWeightKg` to stomach capacity calculations. Both implementations are now correct:

- **Web:** `effectiveDerived = deriveSettings({ ...settings, weightKg: effectiveWeightKg })` → `hourlyRate={effectiveDerived.hourlyRate}` passed to all stomach functions.
- **RN:** `derived = deriveSettings({ ...settings, weightKg: effectiveWeightKg })` → `derived.hourlyRate` used for `stomachCapMilk` and all downstream calls.